UNIT → 2

→ OPERATORS
- Arithematic
- Relational
- Logical
- Bitwise
- Assignment

(!=) → ≠

% → mod → remainder

% 26/3 = (2) → jo remainder bachega

Unary: ++, --
Binary: +, -, *, ÷, %
<, >, <=, >=, ==, !=
&, &&, ||, !
=, +=, -=, *=, %=, /=
Ternary: ? :

Unary operator
Arithematic
Relational
Logical
Assignment
Ternary / Conditiond

If operator comes, is table ke marks hai

- a = 5
  ++a; iska mtlb +1 hojayga
  ans = 6 ayega
  Ily --a; me 4 ayega

} Unary me 1 operands (variable)

→ Ternary me 3 operands chahiye
→ Binary me 2 operands chahiye
- Relational
  Output → 0 or 1 → True, False

a = 5, b = 6

a <= b;	b < a;	b ≥ a;
1	0	1

a == b
0

a = 6, c = 2

a = !c
1

a = 5

→ ++a pre-increment , gives 6
→ a++ post-incoement , gives 5

→ If a = 5 b = 6
if (a<=5 && b<9)

AND
(&&) → both cond true honi chahiye

{ ——— ——— } True h to cond print hojaygi
}

OR
(||) → dono me se 1 bhi sahi hai to print hojayga

(!) → NOT if (!a)
agr a = 0 hai sirf tab hi true hoyega.

1 gives → 0
0 → 1

→ Ex of C operators
• Arithematic

```
# include <stdio.h>
int main ()
{
  int a=25 , b=5;
print f ("a+b = %. d\n", a+b);
print f ("a-b = %. d/n", a-b);
print f ("a*b = %. d/n", a*b);
print f ("a/b = %. d/n", a/b);
print f ("a%. b = %. d/n", a%. b);
print f ("+a = %. d/n" , +a);
Print f ("-a = %. d/n" , -a);
Print f ("a++ = %. d/n", a++);
```

```
Printf("a-- = %d/n", a--);
return 0;
}
```

Outputs :-

a+b = 30 a%b = 0 a-- = 26
a-b = 20 +a = 25
a*b = 125 -a = -25
a/b = 5 a++ = 25

- Relational Operators

0, 1 →
1
False

```
# include <stdio.h>
int main ()
{
 int a=25 , b=5 ;
printf ("a<b, %d/n", a<b);
printf ("a>b, %d/n", a>b);
printf ("a<=b, %d/n", a<=b);
printf ("a>=b, %d/n", a>=b);
printf ("a==b, %d/n", a==b);
printf ("a!=b, %d/n", a!=b);
return 0;
}
```

Outputs :-

a<b → 0 a==b → 0
a>b → 1 a!=b → 1
a<=b → 0
a>=b → 1

→ AND operator (&&)

- True if both operands are true

eg :-

```
#include <stdio.h>
int main()
{
 int a=5, b=10;
if (a>0 && b>0);
print f ("Both a & b are +ve numbers. /n");
 return 0
}
```

Both True
Run Hojayga

→ OR OPERATOR (||)

- True if atleast 1 operand is true

eg :-

```
#include <stdio.h>
int main()
{
 int a = -5, b=10;
if (a>0 || b>0);
print f ("Atleast one of a or b is the no. /n");
return 0
  }
```

1 true h, Run Hojayga

→ NOT OPERATOR (!)

- Returns true if operand is false & vice versa

Ex

```
int a=0
if (!a) {
print f ("a is zero. /n");
   }
```

→ Expression
Combination of operators, constants & variable
Result = a+b - (a*c)

→ ARITHEMATIC OPERATOR PRECEDENCE

- In C, follow precedence order which determines the sequence in which operation are performed
- Parenthesis > Multiply or divide > Add or subt
  ( ) > {*} = {÷} > {+} = {-}

Eg :-

```
# include <stdio.h>
int main () {
int a=10, b=5, c=2, result;
result = a+b* c- (a/b)%. c;
print f ("Result : %.d/n", result);
return 0;
}
```

Process →
a+b * c -(a/b) % c
a+b * c - 2 % c
a+b * c - 0
a+10
20

→ Bitwise Operator

Operator	Name
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT
<<	Bitwise Left shift
>>	Bitwise Right shift

→ Binary Conversion
16 8 4 2 1 } jahi tk lijao 2^n krke
For eg 3
3 = 2+1 to 2 aur 1 ke niche 1
baki sare 0
⇒ 3 = 0 0 0 1 1

→ Bitwise AND
a = 3 b = 5
a = 0011 b = 0101
0011 } dono 1 hui toh hi 1 milega
x 0101
AND → 0001

→ OR ke case me koi bhi ek 1 hui to 1 ayega
eg 0011
0101
OR → 0111

→ XOR jb a aur b mein ulte h to 1 varna 0 0011
0101
XOR → 0110

⇝ NOT (~)

a = ~~0011~~ 0101 0 → 1

~a = ~~1100~~ 1010 1 → 0

→ Left Shift

Left ko hatake right me 0

<< a = 1010

→ Right Shift

Right most ko hatake left me 0

>> a = 0100

→ Ternary operators

Syntax

condition ? expression_if_true : expression_if_false

if cond → True → exp_if_true prints

→ False → exp_if_false prints

→ Eg :-

```
# include <stdio.h>
int main () {
   int a = 10 , b = 20;
int max = (a > b) ? a : b;
print f ("Max = %d \n", max) ; //Output : Max = 20
return 0;
  }
```

a < b

exp → false

so, b print hoyega

isLie b ko rightside likha aur a ko left.

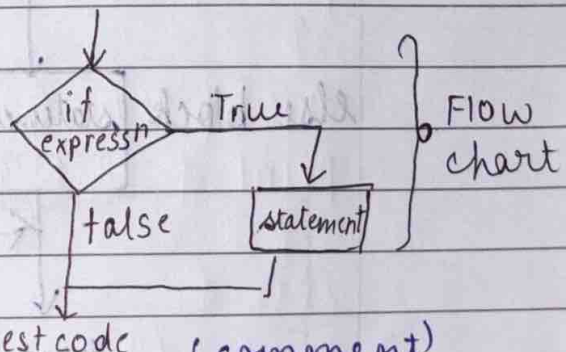
__/__/__

→ control statement
Dictates flow of execution in program, allows to make decisions, repeat tasks, or jump to specific parts of code.

control statements

conditional	Looping	Jumping
if statement	for loop	break statem
if else statement	while loop	continue
nested if else	do while loop	go to
else if construct		
switch statement		

→ CONDITIONAL STATEMENT
• if statement
→ always used with condition
→ single condition
→ Syntax :-
if (condition) ⓘ mahi ayega () bracket
{
Statements;
}

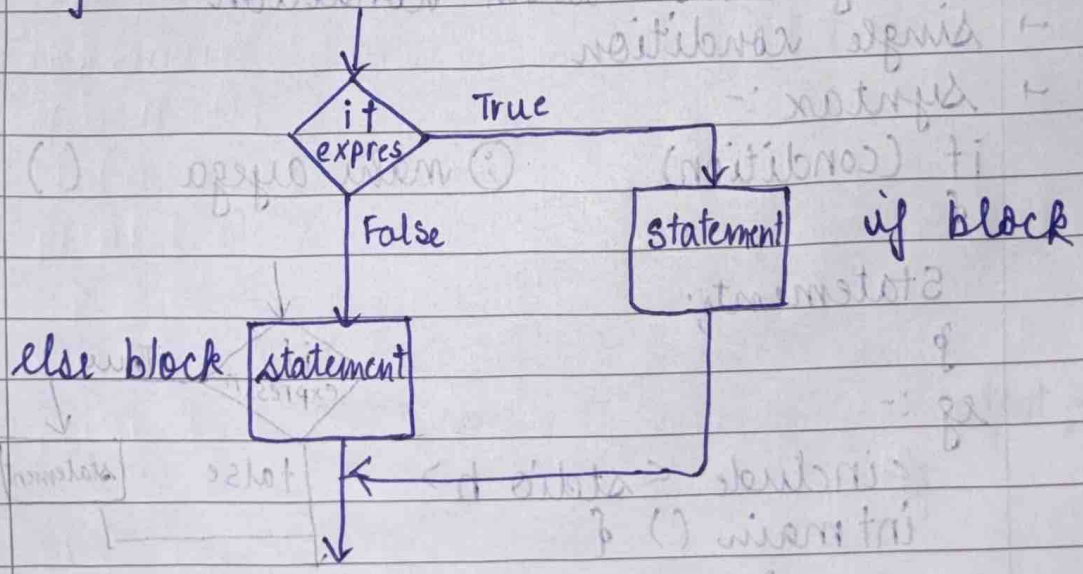


eg :-
#include <stdio.h>
int main () {
int number = 10 (comment)
// if statement → Ye likne ki zrurt ni h
if (number > 5) {
print f ("The number is greater than 5.\n");
}

→ Types of if statement
- if else
- nested if else
- else if ladder

→ IF ELSE STATEMENT → IF block
- 2 blocks of statement → ELSE block
- If ✓ True → IF block executes
  False → ELSE block executes
- Syntax

```
if (condition)
{
 True Block of statements
}
else
{
 False block of statements
}
```



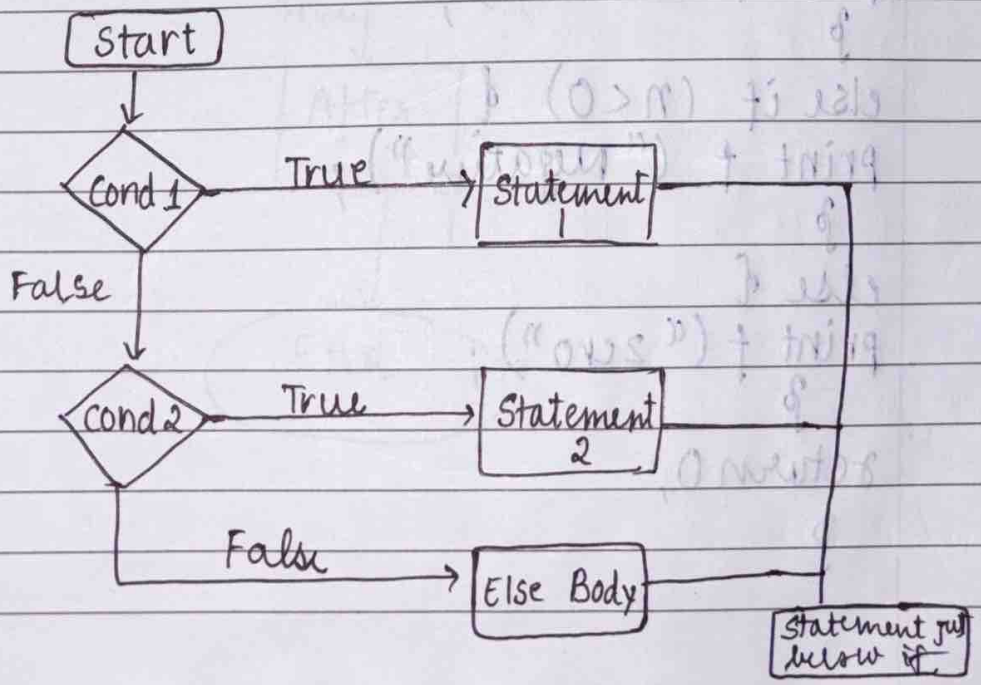
→ eg →

```
#include <stdio.h>
int main () {
 int number;
printf ("enter an integer");
scanf ("%d", &number);
if (number %.2 == 0) {
 printf("%.d is even.\n", number);
    }
else {
 printf ("%.d is odd.\n", number);
  }
 return 0;
 }
```

→ ELSE IF LADDER

- Multiple conditions
- when condition is true, the corresponding block of code is activated & rest is bypassed

If none is true, else block executed



→ Syntax

```
if (condition 1)
{ statement1 (execute if c1 is true)
  }
else if (condition 2)
{ statement 2 (executes if c2 is true)
  }
else if (condition 3)
{ statement 3 (executes is c3 is true)
  }
.
.
.
else {
  }
```

→ Example

```
int main () {
int n = 0;
if (n > 0) {
printf ("Positive");
  }
else if (n < 0) {
printf ("Negative");
  }
else {
printf ("zero");
  }
return 0;
  }
```

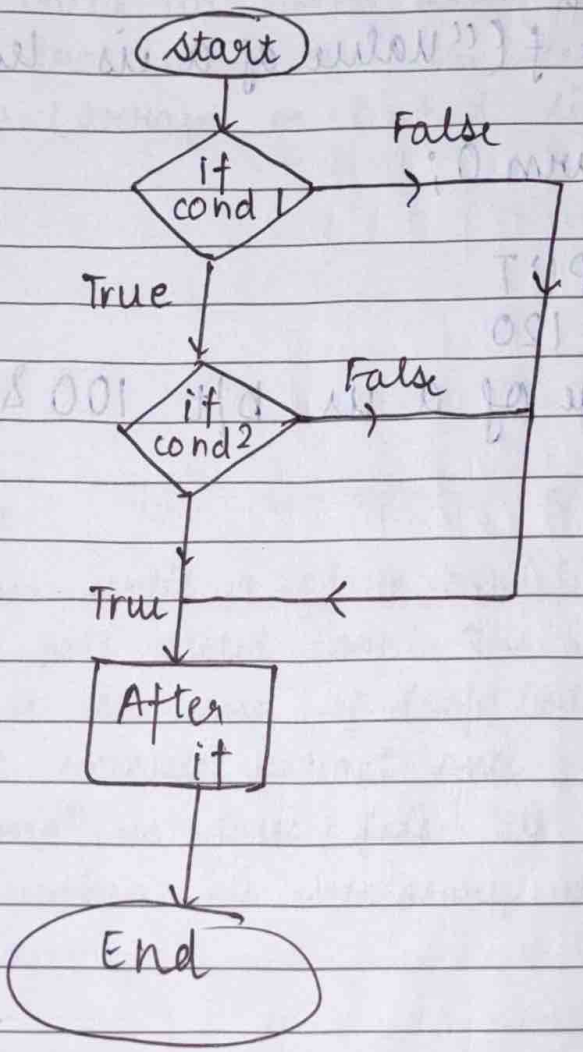
__/__/____

→ NESTED IF

• statement which is target of another statement

Syntax :-

```
if (condition) {
  if (condition 2) { (statement inside nested if)
      }
  else { (statement inside nested else)
     }
  }
else {(statement inside else)
  }
```



eg

```
int main () {
    int a = 120;
    printf ("value of a is : %d\n", a);
    if (a < 200) {
    printf ("value of a is b/t 100 & 200");
    }
    else {
    printf (" value of a is more than 200");
    }
}
else{
printf ("value of a is less than 100\n");
}
return 0;
}
```

→ OUTPUT

a = 120

value of a is b/t 100 & 200

Loops

Looping is defined as repeating same process multiple times until specific condition satisfies it simplifies complex problems into easy ones A loop statement allows to execute a statement multiple times without repetition of code.

→ ADVANTAGES

- Provides code reusability
- We don't need to write same code again & again
- We can transverse over the elements of data structures (arrays or linked lists.

→ TYPES

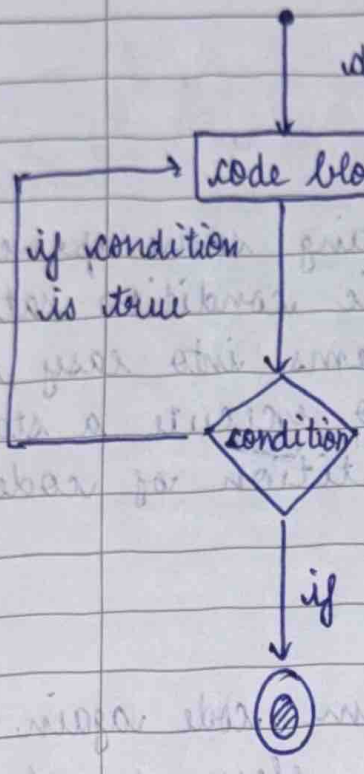
- do while
- while
- for

→ DO WHILE

It continues until a given condition satisfies. It's also called post tested loop. The test condition is evaluated at the end of loop body. The loop body will execute atleast once irrespective of whether cond" is true / false. It's exit controlled loop; used when it's necessary to execute loop atleast once.

- SYNTAX

```
do {code to be executed
} while (condition);
```



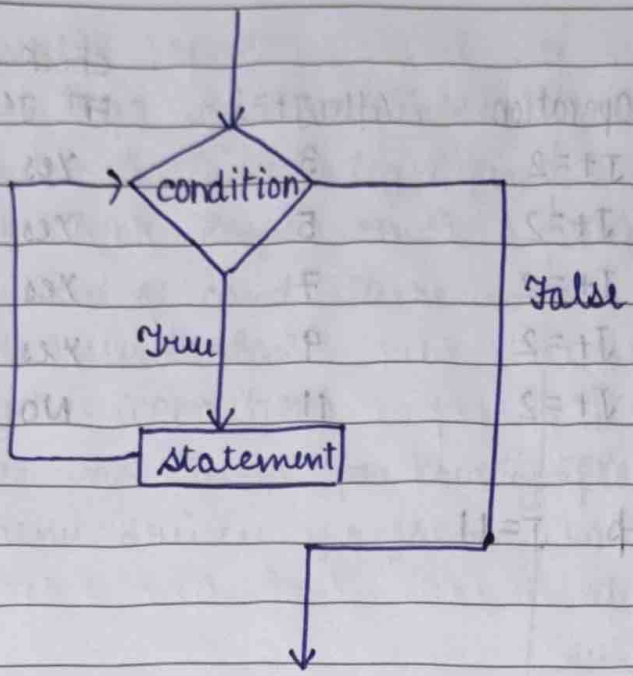

```
#include <stdio.h>
int main () {
int a = 1;
do {
print f ("Hello World \n");
 a++;
} while (a <= 5);
print f ("End of loop");
return 0;
}
```

→ WHILE LOOP. (Pre tested loop)

It allows a part of code to execute multiple times depending upon given boolean condition. It can be viewed as repeating if statement.

- SYNTAX

```
While (condition)
{ code to be executed
}
```



Eg-1

```
int main ()
{
 int i=0;
while (i<50)
{
 printf ("hello-world");
 i = i+1;
}
}
```

Eg-2

```
# include <stdio.h>
void main ()
{ int J=1;
while (J+=2, J<=10)
{ printf ("%d", J);
}
print f ("%d", J);
}
```

J+=2 means J+2
if J=1
J+=2 ⇔ 1+2 = 3

→ outputs

Step	Before Loop	Operation	After J+=2.	check J≤10	Print
1	J=1	J+=2	3	Yes	3
2	J=3	J+=2	5	Yes	5
3	J=5	J+=2	7	Yes	7
4	J=7	J+=2	9	Yes	9
5	J=9	J+=2	11	NO	doesn't Print

After the loop J=11 prints 11

⇒ Output → 3, 5, 7, 9, 11

→ INFINITIVE WHILE LOOP

If the expression passed in while loop results in any non zero value then loop runs ∞ times

```
while (1)
{ statement
}
```

→ WHILE LOOP	DO WHILE LOOP
1. It's an entry control loop because firstly cond^n is checked then loop body is executed	It's an exit control loop ∵ firstly body is executed then cond^n is checked true / false
2. It's statement may not be executed at all	It's statement executes atleast once
3. It terminates when the cond becomes false	As long as cond^n is true compiler keeps executing do while loop.

__/__/____

WHILE LOOP	DO WHILE LOOP
4. The test cond variable must be initialized first to check loop's cond".	4. The variable of test cond is initialised in the loop also.
5. In end of cond there is no semicolon while (condition)	5. In end of cond there is semicolon & while (condition);
6. It's not used for creating menu driven programs	6. It's mostly used for creating menu driven programs :: at least once, loop executes whether cond true / false.
7. The no of executions depend on cond defined in while block	7. Irrespective of cond" mentioned min of 1 execution occur

→ FOR LOOP

```
for (initialisation; check/test expression ; updation)
{
  body consisting multiple statements
}
```

The for loop is entry control loop that executes the statements till given condition. All elements (initialisation, test cond, increment) are placed together to form a for loop inside parenthesis with 'for' keyword. It's used to transverse arrays.

It follows a very structured approach where it begins with initialising a condition and updating statements as a part of it's syntax.

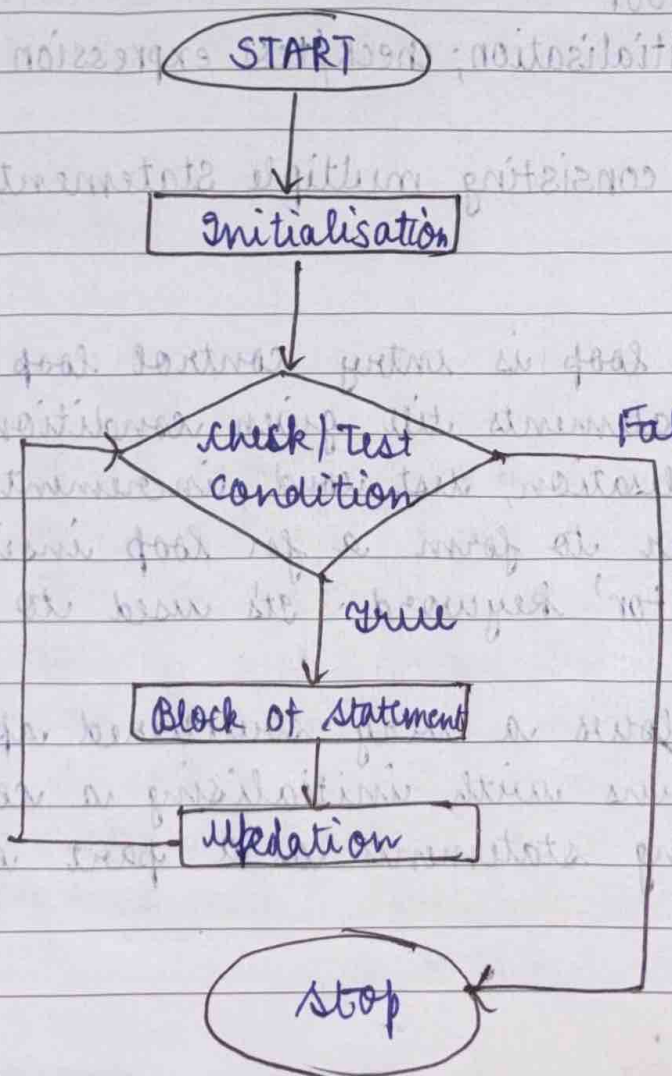
STEP 1 Initialization is the basic step of for loop. This occurs only once during start of loop. During initialization, variables are declared or already existing variables are assigned some values.

STEP 2 Condition statements are checked & only if the cond" satisfies the loop, we can continue otherwise loop broken.

STEP 3 All statements inside loop are executed

STEP 4 Updating the values of variable has been done as defined in loop.

STEP 2 CONTINUES UNTIL LOOP BREAKS.



__/__/____

```
# include <stdio.h>
int main () {
int num, i;
printf ("Enter a number");
scanf ("%d", &num);
for (i=1; i<=10; i++) {
printf ("%d * %d = %d\n", num, i, num*i);
}
return 0;
}
```

→ INFINITE FOR LOOP

```
# include <stdio.h>
int main ()
{
for ( ; ; )
{ printf ("LUV MALIK");
}
return 0
}
```

→ No cond ⟶ ∞ times

→ NESTED FOR LOOP

```
# include <stdio.h>
int main () {
int i, J;
for (i=1; i<=5; i++) {
for (J= 1; J<=i ; J++) {
printf ("%d", J);
}
printf ("\n");
}
return 0;
}
```

→ SWITCH CASE

It's a flow control statement in which we can define a switch variable & then execute different codes based on the value of switch variable. It's alternative of else if ladder.

It evaluates a given expression based on the evaluated value (matching certain condition) it executes statements associated with it. Basically it's used to perform different actions based on diff cond.

- SYNTAX

```
switch (expression)
{
 case value 1 : statement – 1;
   break;
 case value 2 : statement – 2;
   break;
   -
   -
   -
 case value n : statement – n;
   break;
 default : default – statement;
```

- Eg

```
#include <stdio.h>
int main ()
{ int var = 3;
  switch (var)
{ case1 :
 printf ("case 1 is matched");
```

```
break;
case 2:
printf ("case 2 is matched");
break;
default:
printf ("default statement is matched");
break;
}
return 0;
}
```

→ CONDITIONS OF SWITCH CASE STATEMENTS

- The 'case value' must be of 'char' or 'int' type.
- There can be 1 or N no of cases
- The values in the case must be unique
- Each statement of the case can have a break statement, it's optional
- The default statement is also optional.

ARRAYS

Array is one of the most used data structure in C. Its simple & fast way of storing multiple values under single name.

Its fixed size collection of similar data items stored in contiguous memory locations. It can be used to store collection of primitive data types such as int char, float etc & also derived & user defined data like pointers & structures.

It provides a way to store multiple elements without having to name each one.

- Each element of an array is of same data type & carries same size
- First element is stored at smallest memory location
- Elements can be randomly accessed since we can calculate address of each element with given base address & size.

→ DECLARATION OF ARRAY

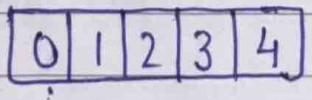
Declared by specifying it's name, the type of it's elements & size of it's dimensions

- SYNTAX

data type – arrayname [size];

int Arr [5];

Arr [5]



0,1,2,3,4 → array indices

__/__/____

→ TYPES

It has 2 types based on it's number of dimens

- 1 D Array
- Multi dimensional Arrays

→ ONE DIMENSIONAL ARRAY

- Have only 1 dimension
- data type — array name — [size]

1	2	3	4	5	6

Eg-1

```
#include <stdio.h>
int main() {
int arr[5];
for (int i = 0; i<5; i++) {
  arr[i] = i*i - 3*i + 1;
  }
printf("Elements of Array");
for (int i=0; i<5; i++) {
  printf("%d", arr[i]);
}
return 0;
  }
```

Output :-

i=0, arr[0] = 0 - 3(0) + 1 = 1

arr[1] = 1 - 3 + 1 = -1

arr[2] = 4 - 6 + 1 = -1

arr[3] = 9 - 9 + 1 = 1

arr[4] = 16 - 12 + 1 = 5

1	-1	-1	1	5

→ Eg-2

```
#include <stdio.h>
int main () {
int arr[5] = {15, 25, 35, 45, 55};
printf ("Element at arr[2]: %d\n", arr[2]);
printf ("Element at arr[4]: %d\n", arr[4]);
return 0;
}
```

output

35 , 55

→ PROGRAM TO PRINT LARGEST & SECOND LARGEST ELEMENT OF ARRAY

```
#include <stdio.h>
void main () {
int arr[100], i, n, largest, second largest;
printf ("Enter the size of array");
scanf ("%d", &n);
printf ("Enter the elements of array");
for (i=0; i<n; i++) {
scanf ("%d", & arr[i]);
}
largest = arr[0];
second largest = arr[i];
for (i=0; i<n; i++) {
if (arr[i] > largest)
{ second largest = largest;
largest = arr[i];
}
else if (arr[i] > second largest && arr[i] != largest)
{ second largest = arr[i];
}
}
```

```
printf ("largest = %d, second largest = %d", largest,
        second largest);
 }
```

→ TWO DIMENSIONAL ARRAY

It has 2 dimensions which can be visualised in the form of rows & columns

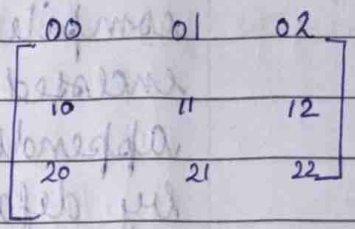
11	12	13
14	15	16
17	18	19

→ SYNTAX

datatype array name [size 1] [size 2]

→ EG-1

```
#include <stdio.h>
void main ()
{
 int arr [3][3], i, j;
for (i=0; i<3; i++)
 {
  for( j=0; j<3; j++)
{ printf ("Enter a [%d][%d]:", i, j);
 scanf ("%d" & arr [i][j]);
 }
 }
printf ("\n printing the elements \n");
for (i=0; i<3; i++)
 { printf ("\n");
for (j=0; j<3, j++)
 { printf ("%d\n", arr [i][j]);
 }
   }
```



a[0][0] = 11
a[0][1] = 12
a[0][2] = 13
a[1][0] = 14
a[1][1] = 15
a[1][2] = 16
a[2][0] = 17
a[2][1] = 18
a[2][2] = 19

Output ↓

11	12	13
14	15	16
17	18	19

STRINGS

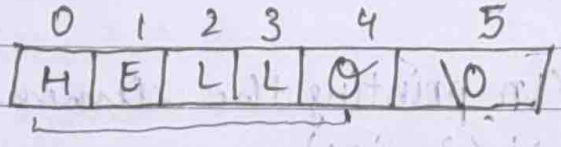
A string is a sequence of characters terminated with a null character \0. When compiler encounters a sequence of characters enclosed in double quotation marks, it appends a null character \0 at the end by default.

↓ used to terminate string

→ DECLARATION

char s[5]

eg → char string[] = "Hello"



```
# include <stdio.h>
int main ()
{
char name [8];
printf ("Enter a string:");
scanf ("%s", &name);
printf ("%s", name);
}
```

IMP ★ If we try to enter LUV MALIK then string me sirf LUV jayega ∵ it's a drawback of scanf func^c ki wo bich me space dekega to sirf phle word ko hi lega.

So, we use fgets() to read line of string & puts() to display.

```
# include <stdio.h>
int main ()
{
char name [80]
printf ("Enter name");
fgets (name, size of (name), stdin);
printf ("Name:")
puts (name);
return 0;
}
```

L	U	V	M	A	L	I	K	\0

L	U	V		M	A	L	I	K	\0

__/__/____

FUNCTIONS

divides large program into building blocks. It is a set of statement that when called, performs specific tasks & contains the set of programming statements inclosed by { }. It can be called multiple times to provide reusability & modularity to program. The collection of function creates a program. Also called subroutines / procedures in other languages.

→ ADVANTAGES

- Avoid rewriting same code again & again
- Can be called multiple times in program from any place in program.
- We can track a large C program easily when divided into multiple func
- Reusability is the main achievement.

→ TYPES

- Standard Library Function
  → Built in functions, defined in header files
  eg → printf(), scanf(), sqrt()
- User defined Function
  you can also create func as per need & can be created by user.

Parameters
~~Arguements~~

→ eg :-

```
int add (int a, int b);   → Func Declaration
int main () {
int num1 = 5, num2 = 3, sum;
sum = add (num1, num2);   → Func Calling
printf ("The sum is : %d \n", sum);
return 0;                 arguements
}
```

→ Func Defn

```
int add (int a, int b) {
    return a+b;
}
```

ye pura yha aayega iye function ki defination hai

→ Function declaration
Provide func name, return type, number & type of it's parameters

return type – name of func (parameter-1, parameter-2)

It can be with / without declaration
int sum (int a, int b);
int sum (int, int);

→ Function defination
return type – func name (para1-type para1-name, para2-type para2-name)

→ Parameters & Arguements

Parameters are variables listed in function definition. They're like placeholders for the values func will use

Arguements are actual values we pass to function when we call it

→ Type of Function

1. Function with arguements & return value

```
int function(int);
function(x);
int function(int x)
{ statements;
  return x;
}
```

2. Function with arguement but no return value

```
void function(int);
function(x);
void function(int x)
{ statements;
}
```

CALL BY VALUE

eg → SWAP TWO NUMBERS

```
#include <stdio.h>
void swap(int,int);
int main() {
  int a=10,
  int b=20,
printf("Before swapping the values in
  main a=%d, b=%d \n", a, b);
  swap(a,b);
```

printf("After swapping values in main a = %d
b = %d \n", a, b);
}
void swap (int a, int b)
{
int temp;
temp = a;
a = b;
b = temp;
printf("After swapping values in function
a = %d, b = %d \n", a, b);
}

3. Function with no argument & no return value #include <stdio.h>
int main ()
{ void print ();
print ();
return 0;
}
void print (void)
{ for (int i = 1; i <= 5; i++)
printf("* \n");
}

Output :-

*
*
*
*
*

4. Function with no arguement but returns a value

```
int add();
int main() {
 int a;
a = add();
printf("\n Total : %d", a);
  return 0;
  }
int add() {
 int a, b;
printf("Enter value of A & B :");
 scanf("%d %d", &a &b);
return a+b;
 }
```

UNIT ~ 3